Missing Account Lockout & Timeout on Authentication Endpoint – Unrestricted Password Guessing

Severity: High

CVSS Score: 7.8 (High)

(Note: This score reflects the worst-case scenario where an attacker successfully guesses valid credentials. Password complexity and MFA status of the target accounts will affect the effective exploitability. If MFA is mandatory for all users, the practical impact is reduced to Medium.)

CWE

- CWE-307: Improper Restriction of Excessive Authentication Attempts
- CWE-799: Improper Control of Interaction Frequency
- CWE-307: Improper Restriction of Excessive Authentication Attempts (Authentication)

Affected Asset

- URL: https://crm-stage.crusource.com – Authentication endpoint (e.g., POST /api/v1/auth/login)
- Parameter: Login credentials (username/email, password)
- Environment: Staging
- Date Discovered: 16-10-2026

Why this is a vulnerability:

- The CRM's authentication endpoint does not enforce any rate limiting, account lockout, progressive delay, or challenge mechanism after repeated failed login attempts. A user or automated client can submit an unlimited number of incorrect password guesses against the same account without triggering any defensive response.
- During testing, multiple consecutive failed login attempts were submitted with incorrect credentials against a test account. Every attempt returned the same response code and equivalent response body. No 429 Too Many Requests status was returned, no Retry-After header was present, no account lockout message appeared, and no progressive delay between attempts was observed.
- This means the login endpoint can be used as a password-guessing oracle at full speed. While the application enforces password complexity at registration, and rejects known-compromised passwords using a database. This is a unique layer of security and

a positive control that reduces the risk of weak passwords from being created. However, an attacker can still take some key words from a user's email, name, designation etc and generate a probable password list to run a credential stuffing attack against the CRM's user base with no restriction from the server.

- The application already detects compromised passwords on registration, which confirms the developers are aware of credential reuse as a risk. However, this protection is only applied at account creation, not at login. An attacker exploiting this gap gains access to the same risk the registration check is designed to prevent.
- The application enforces MFA only during first-time registration or during a password reset. Once an account is registered, subsequent logins require only a username and password. This means a successful password guess immediately grants full account access with no secondary barrier.
- In combination with the missing lockout and rate-limiting controls described above, an attacker can brute-force or credential-stuff any account until a password matches, then log in unimpeded. Even a correct password obtained through a low-volume, slow attack (which would avoid detection under the current setup) leads directly to account takeover.
- Additionally it was noticed that the session expiry timeframe for a set of valid email + password is currently set to 6 months/180 days. Which is an exceedingly long session time limit. This may be a temporary configuration for the staging phase, or could be an oversight altogether. However, in the case of the latter, an immediate action is needed.

Delivery vector (how an attacker would exploit it):

- **Brute Force Against a Single Account:** An attacker identifies a valid email (from a leaked list or a public source) and submits thousands of password guesses per minute. With no lockout, the attacker only needs patience and a wordlist.
- **Credential Stuffing:** The most realistic attack. The attacker collects email:password pairs from a public breach, then submits them against the CRM login endpoint. Because users reuse passwords, a small percentage of attempts will succeed — each one granting full access to a CRM account.
- **Automated Tooling:** Tools like Burp Intruder, hydra, ffuf, or a simple Python script can drive the attack at scale. The absence of a 429 response and Retry-After header means the attacker doesn't even need to slow down.
- **Impact of a Successful Guess:** Full access to the compromised user's CRM data, including customer PII, contacts, deals, notes, and any actions the user is authorized to perform. If the account is an administrator, this escalates to full tenant compromise.
- **Detection Difficulty:** Because failed login attempts are legitimate-looking HTTP requests, they blend into normal traffic. Without rate limiting, logging thresholds, or alerting, an ongoing attack is very difficult to detect.
- **Bypass With No Cost:** The absence of progressive delays means the attacker's only cost is bandwidth. Even a slow attack (e.g., 10 attempts per minute) would rack up tens of thousands of attempts per day.
- **Insider Threat:** One of the most dangerous attack vectors are from within the organization. A low-privileged employee or a contractor with knowledge of one legitimate

email address can attempt to compromise a colleague's account at leisure, from their own device, without any rate-limiting signal to alert the target or the security team. The absence of lockout control means the attacker can keep trying across days or weeks, well below any theoretical detection threshold – because there is no threshold.

Additional Observation

- **Password Policy:** While rejecting passwords found in data breaches is a great control, it was noticed that the complexity has the following rules:

  → *Password length must be at least 8 characters*

  → *Password must include at least one number*

  But it can be improved significantly by adding one or two additional rules such as:

  → *Password must include at least one symbol*

  → *Password must include at least one uppercase and one lowercase letter*

- **Email Notifications:** No email notification was sent to the target account after multiple failed login attempts. Adding this would provide the user an early warning of a brute-force attack.
- **No IP-Based Throttling:** Repeated requests from the same IP were all accepted. Although a brute force attack did not succeed during testing, the fact that the IP was not flagged or timed out shows a lack of security measures. The lack of rate limiting is another issue that is discussed in a separate report.

Remediation Recommendation

1. **Implement account lockout after N failed attempts.** A common policy is to lock the account for 2-5 minutes after 5–10 consecutive failed attempts. Then notify the account owner by email so they can take action and distinguish a lockout from an account compromise.
2. **Implement progressive delays.** After each failed attempt, increase the response time (e.g., start with 1 second, double each time up to a maximum). This slows automated attacks without affecting normal users.
3. **Add rate limiting per IP and per account.** A common policy is to allow 5–10 attempts per minute per IP, and 10–20 attempts per hour per account (the exact number should be discussed and approved by the product stakeholders). Then return 429 Too Many Requests with a Retry-After header when thresholds are exceeded.
4. **Add CAPTCHA after 3 failed attempts.** A CAPTCHA challenge breaks automated attacks and password brute force attacks while allowing humans to continue.

5. **Enable email alerting on failed login bursts.** Notify the account owner when N failed attempts occur within a short window. This gives the user a chance to change their password before a successful guess happens.
6. **Enforce MFA.** This is the single most effective control. Even if an attacker guesses the correct password, MFA prevents account takeover. The MFA is not currently enforced for all users, this is the highest-impact remediation.
7. **Log and monitor failed login patterns.** Track failed attempts per user, per IP, and per user agent. Alert on anomalies (e.g., a single IP targeting multiple accounts, or a single account targeted from multiple IPs).
8. **Apply the same pattern to password reset, MFA verification, and any other authentication-adjacent endpoint.** The above mentioned fixes can also be implemented over email authentication, OTP generation and any other mechanisms to be integrated in future. These are equally attractive to brute-force attackers and adds an additional layer of security by the modern day standards.

Summary

The authentication endpoint does not enforce account lockout, progressive delays, or challenges after repeated failed login attempts. Testing with multiple incorrect password submissions produced no defensive response at any point – no 429, no Retry-After header, no lockout, and no notification to the account owner. This leaves the CRM exposed to brute-force and credential-stuffing attacks, both of which are among the most common attack vectors against SaaS platforms. An attacker with a list of breached email:password pairs or a common password wordlist can automate unlimited login attempts against any account without restriction, and a successful guess grants full access to that user's CRM data including customer PII, deals, and notes. If the compromised account holds administrative privileges, this escalates to full tenant compromise. The application already enforces password complexity and rejects known-compromised passwords at registration, but this protection does not carry over to login. Remediation requires implementing account lockout, progressive delays, per-IP and per-account rate limiting, CAPTCHA after repeated failures, email alerts on failed-login bursts, and most importantly, enforcing MFA across all user accounts.